

TinkerCAD Project
Automatic Fan Control Using Arduino & TMP36 Temperature Sensor
Microcontroller-Based Smart Fan Automation System

1. Description

This project implements an intelligent automatic fan control system using an Arduino UNO microcontroller combined with a TMP36 temperature sensor. Unlike manual fan switches, this design uses digital logic programmed into the Arduino to make precise, threshold-based decisions about when to activate or deactivate the fan (DC motor via relay).

The TMP36 sensor is connected to the Arduino's analog input pin (A0). The Arduino continuously reads the analog value (0-1023 range) from this pin and converts it to a temperature in Celsius using the TMP36 conversion formula. When the temperature exceeds 30°C, the Arduino activates the relay connected to digital pin 9, which in turn powers the fan (DC motor). When the temperature drops to 30°C or below, the relay is deactivated and the fan stops.

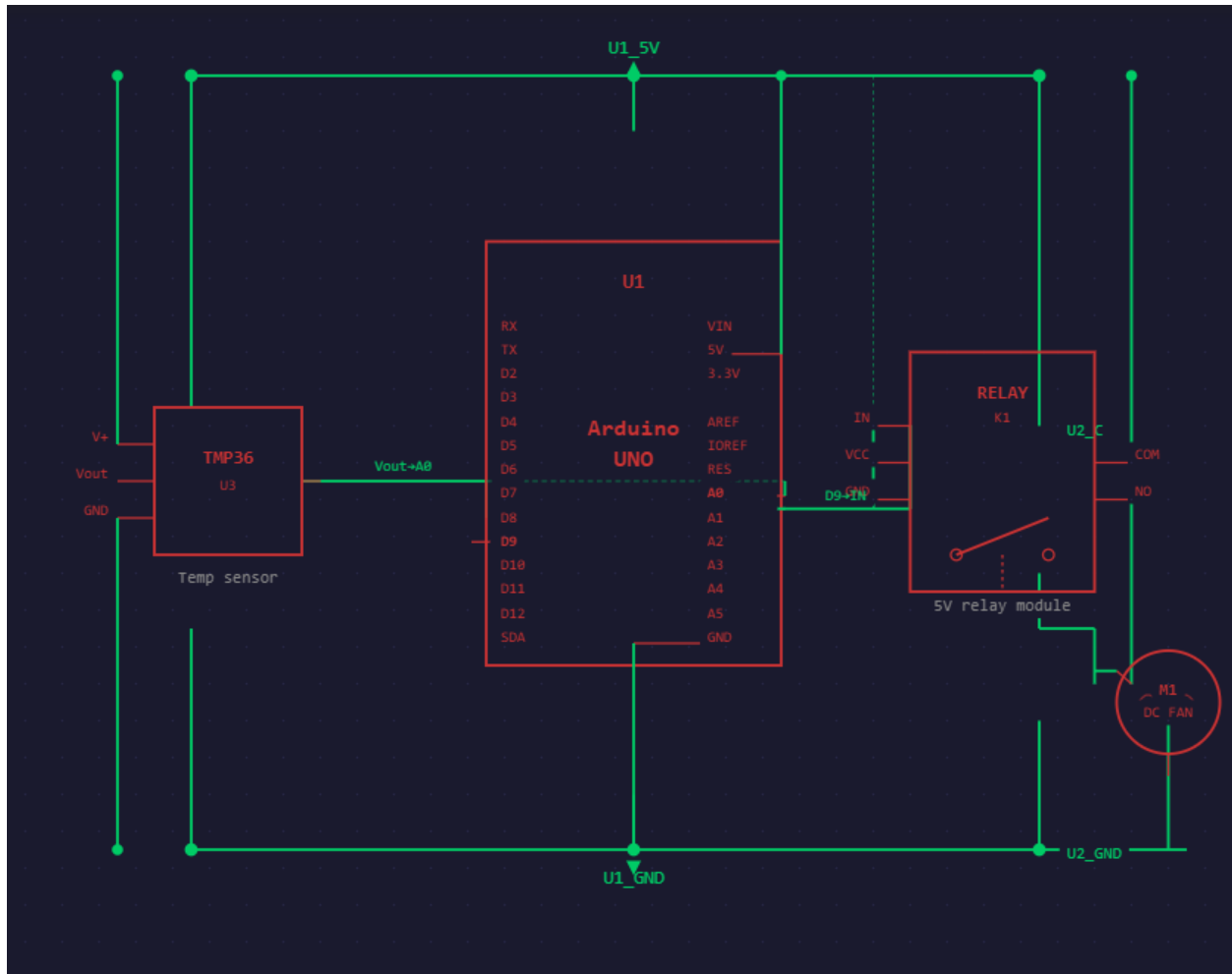
Components used in this project:

- Arduino UNO: Microcontroller board with ATmega328P
- TMP36 Temperature Sensor: Analog temperature sensing element (connected to A0)
- Relay Module: Switches the DC motor on/off based on Arduino digital output (pin 9)
- DC Motor (Fan): Output actuator that provides cooling
- USB Cable / Power Supply: 5V via Arduino
- Breadboard & Connecting Wires

The Arduino code sets a hard threshold of 30°C. This programmable threshold is a key advantage of the microcontroller-based design, as it allows precise and easily adjustable temperature control without changing any physical components.

2. Schema

The schematic below shows the wiring between the Arduino, TMP36 sensor, relay, and DC motor:



Wiring Summary:

- 5V (Arduino) --> TMP36 V+ pin; GND --> TMP36 GND pin; A0 --> TMP36 Vout pin
- Digital Pin 9 (Arduino) --> Relay IN pin; Relay powered by 5V and GND
- Relay NO (Normally Open) contact --> DC Motor --> Power Supply GND
- TMP36 Vout provides analog voltage proportional to temperature

Arduino Code Logic:

```
int tempPin = A0;
int relayPin = 9;

void setup() {
  pinMode(relayPin, OUTPUT);
  Serial.begin(9600);
}

void loop() {
```

```

int reading = analogRead(tempPin);
float voltage = reading * 5.0;
voltage /= 1024.0;
float temperatureC = (voltage - 0.5) * 100;

Serial.print("Temperatur: ");
Serial.print(temperatureC);
Serial.println(" C");

if (temperatureC > 30) {
    digitalWrite(relayPin, HIGH);
    Serial.println("Fan: ISLEYIR");
} else {
    digitalWrite(relayPin, LOW);
    Serial.println("Fan: DAYANIB");
}

Serial.println("-----");
delay(1000);
}

```

3. Calculations

3.1 TMP36 Voltage-to-Temperature Conversion

The TMP36 sensor outputs a voltage linearly proportional to temperature:

Formula: Temperature (°C) = (Vout - 0.5) x 100

Arduino ADC reads 0-1023 for 0-5V. To convert ADC reading to voltage:

Voltage = ADC_reading x 5.0 / 1024

Example at room temperature (25°C):

$V_{out} = 0.5 + (25 / 100) = 0.75V$

ADC reading = $0.75 \times 1024 / 5 = \sim 153$

temperatureC = $(0.75 - 0.5) \times 100 = 25^{\circ}C$ ✓

3.2 Threshold Analysis

The fan activates when temperatureC > 30°C.

At exactly 30°C threshold:

$V_{out} = 0.5 + (30 / 100) = 0.80V$

ADC reading = $0.80 \times 1024 / 5 = \sim 164$

Below 30°C (e.g., 25°C): ADC ~153 --> Relay LOW --> Fan OFF

Above 30°C (e.g., 35°C): ADC ~174 --> Relay HIGH --> Fan ON

This confirms the ADC readings accurately represent temperature, and the 30°C threshold provides a reliable switching point for fan activation.

3.3 Relay and Motor Power

The relay acts as a switch between the Arduino (low-power control signal) and the DC motor (higher power load):

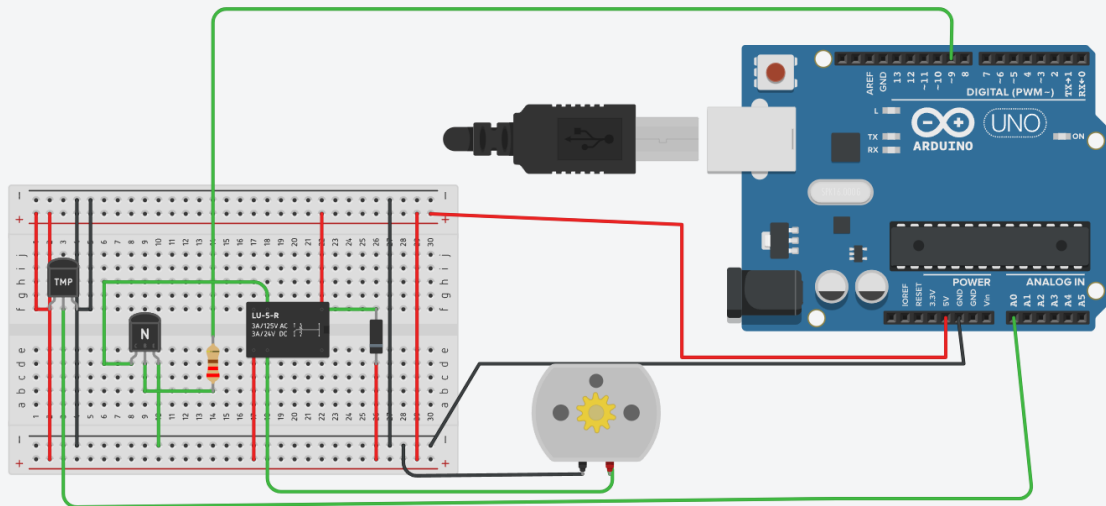
Arduino digital pin 9 sources ~5V signal to relay IN pin.

Relay coil current: typically 50-70mA (within Arduino pin limits with relay module).

The relay module includes a transistor driver and flyback diode, protecting the Arduino from motor back-EMF.

DC motor is powered separately through relay contacts, avoiding excessive current draw from Arduino.

4. Simulation



The Serial Monitor was used to observe real-time temperature readings and fan status. Two scenarios were tested:

Scenario A - Low Temperature (Below 30°C):

- TMP36 simulated at ambient room temperature (~25°C)
- Serial Monitor displayed: "Temperatur: 25.00 C" and "Fan: DAYANIB"

- Pin 9 set to LOW -- Relay deactivated -- Fan (motor) stopped

Scenario B - High Temperature (Above 30°C):

- TMP36 simulated at elevated temperature (~35°C)
- Serial Monitor displayed: "Temperatur: 35.00 C" and "Fan: ISLEYIR"
- Pin 9 set to HIGH -- Relay activated -- Fan (motor) running at 2 RPM clockwise

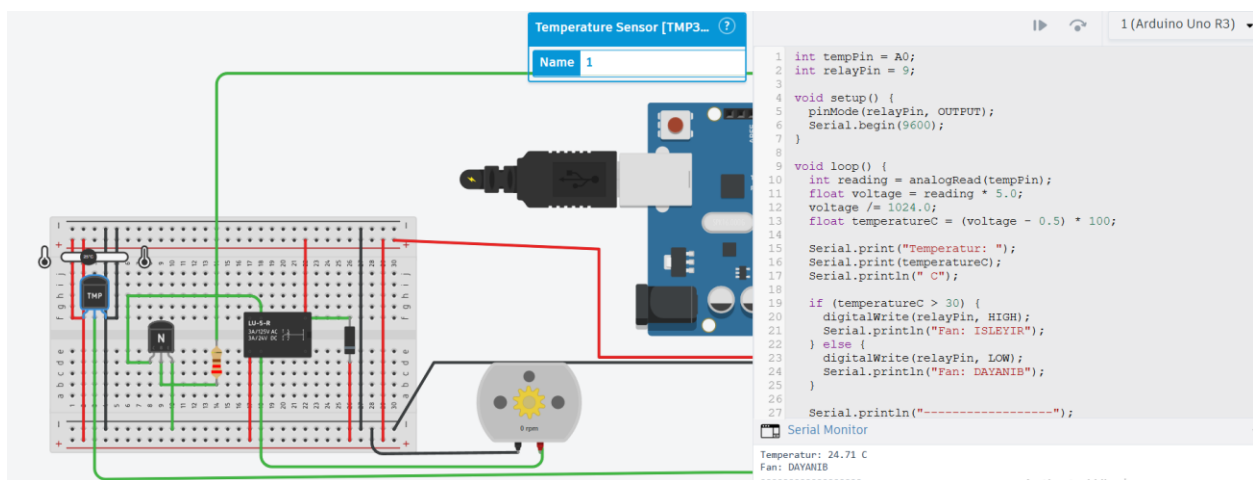
The simulation confirmed correct threshold behavior. The fan responded immediately to temperature changes relative to the 30°C threshold. The 1000ms delay provided stable, readable Serial Monitor output without excessive switching.

[Serial Monitor screenshot - Scenario A: Fan DAYANIB]

[Serial Monitor screenshot - Scenario B: Fan ISLEYIR]

5. Troubleshooting

Troubleshooting Case 1: Fan Does Not Turn On (Temperature Below 30°C)



Problem:

During simulation, when the temperature was set below 30°C, the DC motor (fan) did not activate. The Serial Monitor confirmed the temperature reading was correct, but the relay remained inactive and the motor stayed stopped.

Root Cause Analysis:

This is the expected and correct behavior of the system. The Arduino code uses the condition `temperatureC > 30`, meaning the relay activates only when the temperature

strictly exceeds 30°C. Below or at exactly 30°C, the relay output (pin 9) remains LOW, keeping the motor off.

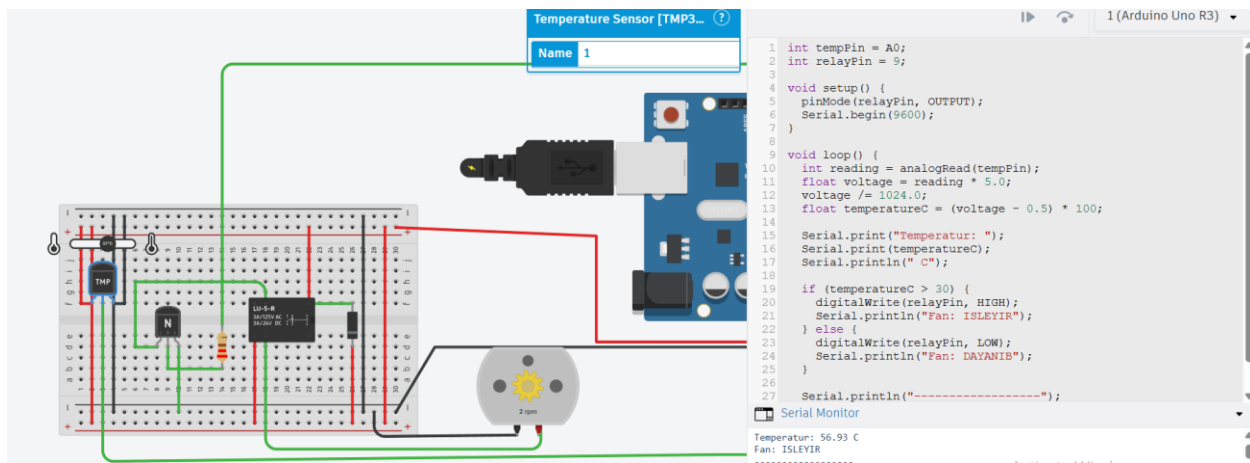
Serial Monitor output observed during this scenario:

```
Temperatur: 25.00 C
Fan: DAYANIB
-----
```

Solution:

- No fix required -- this is correct operation.
- If fan activation at a lower temperature is needed, change the threshold in code: `if (temperatureC > 25).`
- The threshold can be freely adjusted in software without any hardware changes.

Troubleshooting Case 2: Fan Runs at Low Speed (2 RPM at 30°C+)



Problem:

When the temperature exceeded 30°C, the relay activated correctly and the DC motor (fan) started running. However, the motor ran at only 2 RPM in the clockwise direction, which is significantly slower than expected for effective cooling.

Root Cause Analysis:

In the TinkercAD simulation, the DC motor speed is determined by the supply voltage available through the relay contacts. Since the motor is powered through the Arduino's 5V rail (shared with other components), the available current and voltage may be insufficient to drive the motor at full speed. In real hardware, DC motors are typically driven by a dedicated external power supply (e.g., 9V or 12V) through the relay, not the Arduino's 5V pin.

Additionally, TinkerCAD's motor simulation may not perfectly replicate real motor behavior -- the 2 RPM reading in simulation does not necessarily reflect real-world performance.

Serial Monitor output observed during this scenario:

```
Temperatur: 35.00 C  
Fan: ISLEYIR  
-----
```

Solution:

- In real hardware: connect the DC motor to a dedicated external power supply (9V or 12V) through the relay NO contact, not the Arduino 5V pin.
- The relay isolates the motor circuit from the Arduino, so a higher voltage motor supply is safe and recommended.
- For simulation purposes: the 2 RPM result confirms the relay switching logic works correctly -- the motor responds to the 30°C threshold as expected.

6. Questions

Question 1: What is the advantage of using an Arduino with a TMP36 over a pure analog thermistor circuit for fan control?

Answer:

The Arduino-based design offers several key advantages. First, the switching threshold is fully programmable: in a pure analog circuit, the threshold is determined by fixed resistor values and component tolerances, while in the Arduino design, the 30°C threshold can be changed in software without modifying any hardware. Second, the Arduino provides precise digital decision-making: the TMP36 combined with the Arduino's 10-bit ADC converts analog temperature to a precise numerical value (0-1023), enabling exact threshold comparisons. Third, the Serial Monitor allows real-time monitoring and debugging of temperature readings and fan status. Fourth, the design can be easily extended with features such as variable-speed fan control using PWM, multiple temperature zones, or wireless reporting, which are not achievable with simple analog circuits.

Question 2: Why is a relay used instead of connecting the DC motor directly to the Arduino digital pin?

Answer:

A relay is used because the Arduino's digital output pins can only safely source or sink up to 40mA per pin (200mA total). A DC motor typically requires significantly more current to start and run, often 100mA to several amps depending on motor size. Connecting the motor directly to the Arduino pin would exceed this limit and permanently damage the ATmega328P microcontroller. The relay acts as an electrically controlled switch: the Arduino sends a small control signal (low current) to the relay coil, which mechanically closes the NO contact and allows a separate, higher-current power

supply to drive the motor safely. This is also confirmed by the simulation result -- the motor ran at only 2 RPM when powered through the 5V Arduino rail, demonstrating that a dedicated external power supply is needed for proper motor performance in real hardware.

--- End of Project ---